Highlights

HI-MGN-V: Generative hierarchical interpolating MeshGraphNets for the scatter of structural response fields caused by unmeasured imperfections

SiHun Lee, Dong-Hyuk Park, Taesoo Bang, Seung-Hoon Kang

- HI-MGN-V generates complete response fields whose scatter comes from an unmeasured input.
- A latent variable modulates every block of a hierarchical MeshGraphNets processor.
- Global, local and hybrid latent placements are one setting of one implementation.
- A flow-matching prior samples on a coarse latent instead of on the fine finite element mesh.

HI-MGN-V: Generative hierarchical interpolating MeshGraphNets for the scatter of structural response fields caused by unmeasured imperfections

SiHun Lee[a], Dong-Hyuk Park[b], Taesoo Bang[b], Seung-Hoon Kang[b,*]

[a]*Mobile Experience, Digital Twin AX Lab., Samsung Electronics Co., 129, Samsung-ro, Yeongtong-gu, Suwon-si, 16677, Gyeonggi-do, Republic of Korea*
[b]*Department of Aerospace system Engineering, Sejong University, 209, Neungdong-ro, Gwangjin-gu, Seoul, 05006, Republic of Korea*

Abstract

Graph neural network (GNN) surrogates such as MeshGraphNets predict one response field for one input mesh. Manufactured structures, however, often have inputs that are not measured, such as small geometric imperfections, and one nominal design then gives a scatter of response fields instead of one field. For an imperfection-sensitive structure such as an axially compressed cylindrical shell, the same nominal design either stays unbuckled or buckles, depending on the imperfection. A deterministic surrogate trained by mean squared error returns the average of the two states, which is not a response that any analysis produces.

To generate this scatter, we propose HI-MGN-V, a generative extension of Hierarchical Interpolating MeshGraphNets (HI-MGN). The hierarchical V-cycle processor of HI-MGN is kept, and a latent variable modulates the node features before every message passing block through a gated feature-wise linear modulation. During training, the latent variable is inferred from the finite element (FE) result by a graph encoder and regularized by a maximum mean discrepancy; at prediction time it is drawn from a flow-matching prior conditioned on the nominal design only. The latent variable can be one vector per structure (global), a field on the coarsest clusters of the hierarchy (local), or both (hybrid), selected by a single setting. Sampling iterates on a

*Corresponding author

Email addresses: s.hun.lee@samsung.com (SiHun Lee), shkang@sejong.ac.kr (Seung-Hoon Kang)

96-dimensional vector or on 256 coarse clusters instead of on the FE mesh, so every generated field costs one forward pass of the graph network.

The model is evaluated on a shell-buckling dataset of 150 cylinders, each analysed 64 times with a different random dimple that the model never sees, and on eight geometries outside the training range with 256 FE analyses each. The generated scatter of the peak displacement is compared with the FE scatter geometry by geometry and against the value that FE results alone reach with the same number of samples. The deterministic HI-MGN and ports of two published generative mesh surrogates, LDGN and SAR, are compared under the same protocol.

Keywords: Graph Neural Network, Mesh-based Simulation, Generative surrogate model, Geometric imperfection, Shell buckling, Flow matching

---

1. Introduction

As the fields of artificial intelligence (AI) and machine learning (ML) have progressed rapidly, efforts to build ML-based physics surrogate models have also increased. Earlier methods mainly applied classical regressors such as Gaussian process regression and radial basis function regression to linearly reduced subspaces such as proper orthogonal decomposition [1, 2, 3]. Following these regressors, feed-forward neural networks [4, 5], long short-term memory networks [6], and variational autoencoders (VAEs) [7, 8] were considered upon the linear subspace. As ML methods improved, fully data-driven nonlinear models such as autoencoders [9, 10] and VAEs [11, 12, 13] were used as physical surrogate models. While these methods have been successful on structured grids or fixed discretizations, they can quickly lose effectiveness on unstructured meshes and on geometry-varying problems.

Mesh-agnostic methods have changed this landscape by allowing different meshes for training and inference. Point-set networks such as PointNet and PointNet++ were among the first to be used as mesh-agnostic surrogate models [14, 15]. Operator-learning methods approximate the solution operator that maps input functions to output functions: DeepONet and its geometry-aware extensions [16, 17], Geo-FNO, which maps an irregular domain to a latent regular one [18], and geometry-informed neural operators [19]. Transformer-based solvers such as Transolver communicate through learned physical slices instead of explicit connectivity [20], at a computational cost that can be large compared with existing methods [21]. As meshes in

numerical simulations are naturally graphs, GNNs that treat the simulation mesh as a graph have also been investigated, starting from MeshGraphNets (MGN) [22]. Since message passing propagates information only one hop per block, multiscale graph methods such as MultiScale MGN and the Bi-Stride Multi-Scale GNN (BSMS-GNN) [23, 24], partitioning approaches such as X-MeshGraphNet [25], and global processors and graph rewiring such as MeshGraphNet-Transformer and PIORF [26, 27] have been proposed to improve long-range interaction. In our previous work, Hierarchical Interpolating MeshGraphNets (HI-MGN) built the hierarchy from the input mesh alone by farthest-point sampling (FPS) and Voronoi clustering, and restored the fine resolution with a learned geometry-aware interpolation [28].

All of these surrogates learn a one-to-one map: one input mesh with its boundary conditions gives one response field. This assumption holds when every input of the analysis is known. In practice, a manufactured structure has inputs that are not measured, such as small deviations of its shape from the drawing, and the same nominal design then gives a scatter of responses, $\mathbf{y} = S(c, \xi)$, with c the nominal input and ξ the unmeasured one. Thin cylindrical shells under axial compression are the classical case: their buckling load is highly sensitive to small geometric imperfections [29], and near the critical load one nominal shell stays unbuckled under one imperfection and buckles under another. A deterministic surrogate trained by the mean squared error (MSE) converges to the conditional mean of $\mathbf{y}$ given c. When the responses form two clusters, the mean lies between them and is not a response that any analysis produces. The surrogate must therefore generate the scatter itself, field by field, from the nominal input only.

Surrogates of input scatter have been studied mainly for scalar or low-dimensional outputs. Stochastic emulators model the response distribution of a scalar quantity [30, 31, 32], and engression learns a generative regression with a scoring rule [33]. For fields, conditional VAEs and conditional GANs were applied to random initial conditions of the Burgers equation [34] and to the load scatter of plates and hulls [35], and mixture density networks to imperfections and initial conditions of mechanical systems [36]. Gen-Fab and GUST generate as-fabricated photonic devices and printed unit cells on pixel grids [37, 38]. Equivariant flow matching [39] and Bi-FORK [40] generate symmetry-breaking and bifurcating solutions. These works either predict scalar or homogenized quantities, work on structured grids, or reproduce a bifurcation of one nominal problem instead of the scatter of an unmeasured input over many designs.

Generative surrogates on unstructured meshes have mostly been developed for turbulent flows. Latent diffusion graph networks (LDGN) compress a flow field by a multiscale graph VAE and generate it by a diffusion model on the coarse latent graph [41], and the scale-autoregressive model SAR generates the latent field scale by scale with flow matching [42]. PIFM-GN, GenCFD, turbulence and airfoil diffusion models, and probabilistic weather models generate flow states in a similar manner [43, 44, 45, 46, 47, 48, 49], and a diffusion last layer adds generated scatter to a neural operator [50]. Three properties of these models matter for the present problem. First, in most of them the scatter is the evolution of a turbulent state in time, and the reference ensemble consists of states of one long simulation, not of repeated analyses with random inputs. Second, the latent variable of LDGN and SAR *is* the compressed field: the generative model has to produce the whole state on a latent graph, and the decoder only lifts it to the fine mesh. Third, sampling iterates a network on that latent graph, whose size grows with the FE mesh, at every diffusion or integration step. For structural meshes with tens of thousands of nodes, where most of the response is fixed by the nominal design and only a small part is decided by ξ, it is more natural to let a deterministic network compute the response from the design and to generate only the part that ξ changes.

Where the latent variable enters the network and how it is regularized are the two remaining design choices. A *global* latent is one vector per sample, as in conditional VAEs, diffusion autoencoders [51] and latent graph simulators [52, 53]; a *local* latent is attached to positions or regions, as in LDGN, spatially varying modulation [54, 55, 56] and hierarchical segmentation models [57, 58]; and *hybrid* latents combine both [59, 60, 61]. Neither option is ruled out by theory: a global vector can encode a location through an input-dependent expansion [50], and local multiscale computation can reproduce long-range correlation [62]. For regularization, the β-VAE weighs a per-sample Kullback–Leibler (KL) divergence [63], whereas InfoVAE and the Wasserstein autoencoder match the distribution of all latent vectors to the prior through a maximum mean discrepancy (MMD) [64, 65, 66]; regularized and two-stage autoencoders fit a distribution to the learned latent vectors after training [67, 68].

Table 1 positions the related generative works by the cause of their scatter and by three criteria of evaluation. *Cause of scatter* names what produces the spread of the reference data: a random initial condition, load, forcing or imperfection, a bifurcation into several solutions, or the evolution of a tur-

bulent state in time (temporal). *Unobserved input* is met when the scatter is caused by an explicitly stated random input whose realization the model does not receive; the phase of a time series and a branch label are not such inputs. *Several results per input* is met when the data contain several solver results for the same model input, so that the predicted scatter can be compared with the solver scatter input by input. The states of one long simulation count; mirrored copies of one solution and results at different loads do not. Columns (a)–(c) are graded strictly: (a) sample realism is met only when every generated sample is scored quantitatively against the reference, by an error to the nearest reference result, a residual or a validity rate; (b) distribution match is met only when the distribution of whole fields is compared, by a distance between field distributions (Wasserstein, energy distance, MMD), a covariance or a variogram, not by per-node statistics, mode frequencies or spectra alone; and (c) out-of-range inference is met only when checks of the kinds required by (a) and (b) are both repeated on inputs outside the training range. A property that is met in part, or not reported, is marked ✗.

Table 2 compares HI-MGN-V with the mesh-based models that are closest to it in construction. MGN and HI-MGN are deterministic. LDGN and SAR are generative, but their latent variable is the compressed field, and their sampler iterates on a latent graph tied to the FE mesh. In HI-MGN-V, the response is computed by a deterministic V-cycle that sees the nominal design, and the latent variable only modulates its blocks.

In this work, we propose *HI-MGN-V*, a generative extension of HI-MGN that produces a scatter of complete response fields on unstructured FE meshes from the nominal design only. HI-MGN-V preserves the encode–process–decode structure and the hierarchical V-cycle processor of HI-MGN, and adds a latent variable that scales, shifts and gates the node features before every message passing block. During training, a graph encoder infers the latent variable from the FE result, and an MMD keeps the set of inferred latent vectors close to a standard normal distribution. At prediction time, the FE result is not available, and the latent variable is drawn from a flow-matching prior that reads only the nominal design. The latent variable can be one vector per structure (global), a field over the coarsest clusters of the hierarchy (local), or both (hybrid). Each choice states an assumption about how the unmeasured input acts on the response, and all three are one setting of one implementation. Fig. 1 shows training and inference.

The main contribution of this work is a generative mesh-based surrogate in which the response is computed by a deterministic hierarchical graph net-

Work	Method	Cause of scatter	Unobs. input	Several results per input	(a) Sample realism	(b) Distrib. match	(c) Out-of-range inf.
Yang & Perdikaris, Burgers [34]	VAE + GAN	initial condition	✓	✓	✗	✗	✗
CVAE, plate and hull [35]	CVAE, CNN	load	✓	✗	✗	✗	✗
Han & Bahmani [36]	MDN	imperfection, initial condition	✓	✓	✗	✗	✗
Gen-Fab, photonic device [37]	cGAN, U-Net	imperfection	✓	✓	✓	✗	✗
GUST, printed unit cell [38]	diffusion, U-Net	imperfection	✓	✓	✗	✗	✗
Equivariant FM, beam [39]	FM	bifurcation	✗	✗	✗	✓	✗
Bi-FORK, metamaterial [40]	AE + FM	bifurcation	✗	✗	✓	✗	✗
Bi-FORK, Allen–Cahn [40]	AE + FM	initial condition	✓	✓	✓	✗	✗
LDGN, EllipseFlow [41]	VAE + diffusion, GNN	temporal	✗	✓	✓	✓	✓
SAR, EllipseFlow [42]	VAE + FM, GNN	temporal	✗	✓	✓	✓	✓
PIFM-GN, EllipseFlow [43]	FM, GNN	temporal	✗	✓	✓	✗	✗
Turbulence diffusion [45]	diffusion, U-Net	temporal	✗	✓	✗	✗	✗
GenCFD, shear flow [44]	diffusion, U-Net	initial condition	✓	✓	✗	✗	✗
Airfoil diffusion [46]	diffusion, U-Net	temporal	✗	✓	✗	✗	✗
Diffusion last layer [50]	neural operator + diffusion	forcing	✓	✓	✗	✓	✗
This work	VAE + FM, GNN	imperfection	✓	✓	✗	✗	✗

Table 1: Positioning. ✓: met and reported; ✗: not met, met in part, or not reported. Method: the generative model, then the network where the paper fixes one (AE autoencoder, FM flow matching, MDN mixture density network). Gen-Fab and GUST generate the as-fabricated shape, not a response field, and name the fabrication process as the unobserved input without a random parameter; Gen-Fab has replicates for its test designs only, scores each sample by its overlap (IoU) with a fabricated replicate, and compares sets through pixel-intensity histograms; GUST compares its response scatter through scalar homogenized stiffnesses only. Several results per input from the states of one simulation (LDGN, SAR, PIFM-GN, turbulence diffusion) are not independent repetitions of an analysis. The equivariant-FM reference is one solution and its mirror image. The airfoil-diffusion scatter is the pseudo-time of an unconverged solver, not a random input. For this work, (a)–(c) are marked ✗ until the results are reported.

Model	Generative model	Latent variable	Role of the latent variable	Sampling iterates on	Hierarchy
MGN [22]	none	none	–	–	flat
HI-MGN [28]	none	none	–	–	FPS–Voronoi V-cycle
LDGN [41]	VAE + diffusion	field on the coarse latent graph	is the compressed field; the decoder lifts it to the mesh	coarse latent graph, every diffusion step	multiscale VAE
SAR [42]	VAE + FM, scale-autoregressive	field per scale	is the compressed field, generated coarse to fine	every scale, every integration step	mesh coarsening per scale
HI-MGN-V (this work)	encoder + conditional FM prior	global vector, coarse-cluster field, or both	modulates every block of a deterministic V-cycle that reads the nominal design	96-D vector or 256 coarse clusters, 30 Heun steps; one V-cycle pass per field	FPS–Voronoi V-cycle

Table 2: Construction of the closest mesh-based models. FM: flow matching.

work and only the part decided by the unmeasured input is generated. Unlike latent generative models that generate the whole field in a latent space tied to the mesh, HI-MGN-V generates a low-dimensional latent variable that modulates every block of the V-cycle, so the prior iterates on a 96-dimensional vector or on 256 coarse clusters, and every generated field costs one forward pass of the graph network. The global, local and hybrid placements of the latent variable share every other component, which allows the placement to be studied as a controlled ablation. The prior is first trained together with the graph network and then refitted alone on standardized latent vectors of the frozen encoder, which separates fitting the prior from learning the latent space. Throughout the manuscript, we evaluate HI-MGN-V against the deterministic HI-MGN and against our ports of the two closest published generative mesh surrogates, LDGN and SAR, on a shell-buckling dataset in which the unmeasured imperfection decides whether the shell buckles. The generated scatter is compared with the FE scatter geometry by geometry, on unseen geometries inside and outside the training range, and against the value that FE results alone reach with the same number of samples.

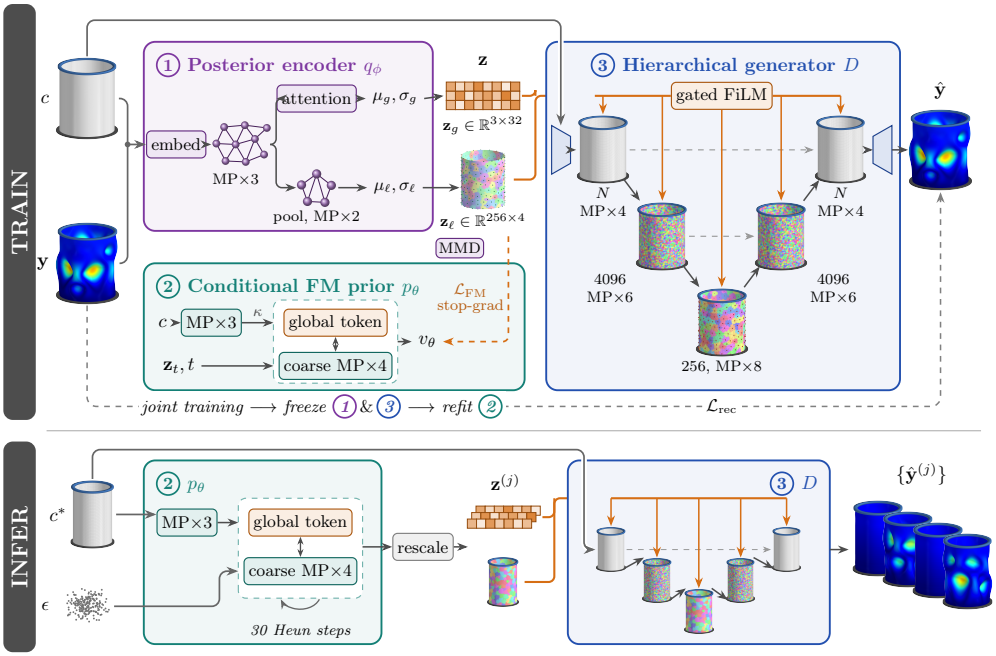


Figure 1: Framework of HI-MGN-V (hybrid placement drawn). Training: the posterior encoder ① q_ϕ infers $\mathbf{z} = (\mathbf{z}_g, \mathbf{z}_\ell)$ from an FE result $\mathbf{y}$ of design c; the hierarchical generator ③ D reconstructs $\mathbf{y}$ from c and $\mathbf{z}$, with $\mathbf{z}$ entering every block of the 28-block V-cycle by gated FiLM; the conditional flow-matching prior ② p_θ learns $\mathbf{z}$ given c from a second encoder draw held fixed (stop-grad). After joint training, ① and ③ are frozen and ② is refitted (Section 2.9). Inference: for a new design c^*, p_θ integrates Gaussian noise ϵ in 30 Heun steps, the result is rescaled to the encoder's latent statistics, and each draw $\mathbf{z}^{(j)}$ gives one field through D, without FE analysis. Images: training geometry $m = 137$ (top) and the geometry with $R = 80\,\text{mm}$, outside the training range (bottom), each with its own FPS–Voronoi hierarchy; all fields drawn, including $\hat{\mathbf{y}}$ and $\hat{\mathbf{y}}^{(j)}$, are FE results (displacements ×3).

2. Methodology

2.1. Model definition

Throughout the model, we follow the basic structure of MeshGraphNets [22] and HI-MGN [28]. A sample is the FE mesh, treated as a graph $G = (V, E)$ with $N = |V|$ nodes and the element edges as bidirectional mesh edges. Each node stores its physical state, the boundary and load conditions, and categorical tags such as boundary-condition types. Each mesh edge stores the relative geometric vector between the connected nodes and its Euclidean norm, $\{dx, dy, dz, \|(dx, dy, dz)\|\}$, evaluated in both the reference and the current configuration, which yields eight-dimensional edge features. World edges for contact are supported by the implementation as in HI-MGN, but the problem considered here has no contact and uses mesh edges only.

We consider a static problem. Given the reference graph G^0, the network predicts the nodal displacement $\mathbf{u}_i \in \mathbb{R}^3$ at the end of loading in one step, without a rollout. The nominal input c consists of the reference coordinates, the shell thickness t, the applied end shortening Δ (constant over nodes, stored per node) and a node type (interior, clamped rim, loaded rim). The unmeasured input ξ enters the solver's initial geometry but not c, and the solver returns $\mathbf{y} = S(c, \xi) \in \mathbb{R}^{N\times 3}$. The training data are M geometries with K FE results each, $\{(c_m, \mathbf{y}_{m,k})\}_{k=1}^{K}$, and the realizations $\xi_{m,k}$ are not given to the model. The goal is a generator

$$\hat{\mathbf{y}} = D(c, \mathbf{z}), \qquad \mathbf{z} \sim p_\theta(\mathbf{z} \mid c), \tag{1}$$

whose output scatter matches $p(\mathbf{y} \mid c)$ on geometries not used in training, including geometries outside the training range. The latent variable $\mathbf{z}$ is not an estimate of ξ: it only has to describe the part of the effect of ξ that is visible in the response field.

2.2. Architecture

HI-MGN-V consists of three networks, as shown in Fig. 2. A hierarchical graph network D maps the nominal input and a latent variable to a displacement field. During training, a posterior encoder $q_\phi(\mathbf{z} \mid c, \mathbf{y})$ infers the latent variable from the FE result. At prediction time, the latent variable is drawn from a conditional prior $p_\theta(\mathbf{z} \mid c)$ that reads only the nominal input. The graph network never sees $\mathbf{y}$, and without the latent variable it is the deterministic HI-MGN.

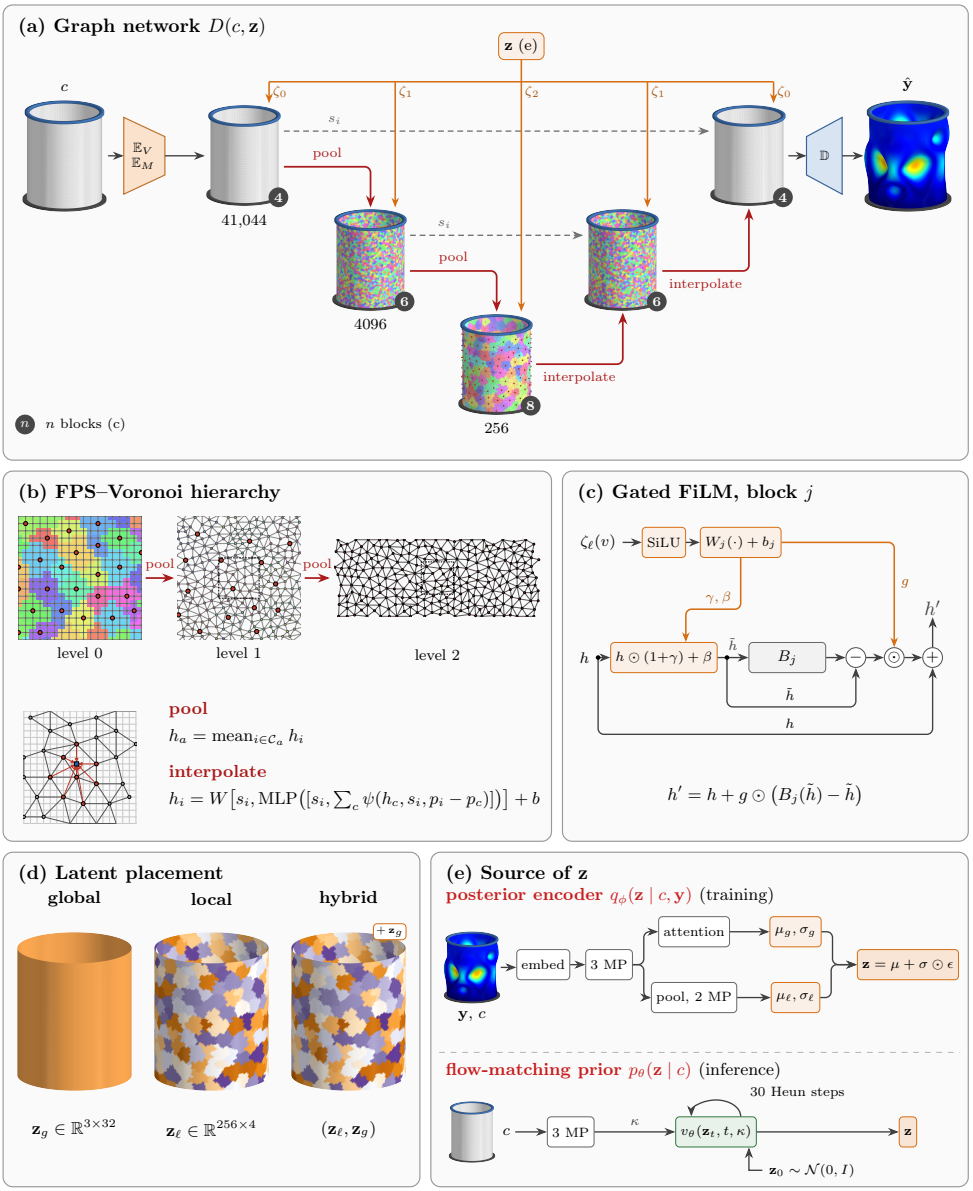


Figure 2: Architecture of HI-MGN-V on the training geometry $m = 137$. (a) Graph network D: the 28-block V-cycle over three levels of one mesh (node counts; badges: blocks), skips s_i, and ζ_ℓ, the part of $\mathbf{z}$ read at level ℓ; the output drawn is an FE result. (b) FPS–Voronoi hierarchy on the unrolled shell (colours: next-level clusters), and the coarse-to-fine graph of one fine node. (c) Gated FiLM of block j; edges are not modulated. (d) Part of $\mathbf{z}$ read by each node. (e) Posterior encoder (training) and flow-matching prior (inference).

The graph network uses the encoder–processor–decoder structure of MeshGraphNets [69, 70, 22]. Throughout the model, multi-layer perceptrons (MLPs) with SiLU activations and LayerNorm, with a latent width of $H = 128$, are used. A nodal encoder $\mathbb{E}_V$ maps each nodal feature to a nodal latent vector h_i,

$$h_i = \mathbb{E}_V(v_i), \qquad v_i = \big[\, \mathbf{u}_i^0 \mid t, \Delta \mid p_i \mid \mathrm{onehot}(\mathrm{type}_i) \,\big], \tag{2}$$

where the state block $\mathbf{u}_i^0$ is zero in the static case, (t, Δ) are input-only condition channels that the model reads but never predicts, and $p_i \in \mathbb{R}^4$ are positional features: the distance to the centroid of the mesh, the mean length of the incident edges, and the return probabilities of random walks of two and four steps on the mesh graph. All continuous inputs and outputs are standardized to zero mean and unit variance with statistics of the training split. A mesh-edge encoder $\mathbb{E}_M$ embeds the edge features,

$$\varepsilon_{ij} = \mathbb{E}_M\big(e_{ij}\big), \qquad e_{ij} = \big[\, \mathbf{d}_{ij}^{\mathrm{cur}}, \|\mathbf{d}_{ij}^{\mathrm{cur}}\|, \mathbf{d}_{ij}^{\mathrm{ref}}, \|\mathbf{d}_{ij}^{\mathrm{ref}}\| \,\big], \tag{3}$$

where $\mathbf{d}_{ij}$ is the relative position of nodes i and j; in the static case the two configurations coincide. A message passing (MP) block first updates the edge latent states from their sender and receiver nodes, and then updates the node latent states from the aggregated incoming messages,

$$\varepsilon_{ij} \leftarrow \varepsilon_{ij} + \phi_M\big(\big[\, h_i,\, h_j,\, \varepsilon_{ij} \,\big]\big), \tag{4}$$

$$h_i \leftarrow h_i + \phi_V\Big(\big[\, h_i,\, \textstyle\sum_j \varepsilon_{ij} \,\big]\Big), \tag{5}$$

where ϕ_M and ϕ_V are independently parameterized MLPs. The decoder $\mathbb{D}$, an MLP without LayerNorm, maps the final fine-level latent state to the normalized displacement, and the prediction is

$$\hat{\mathbf{u}}_i = \mathcal{N}^{-1}\big(\mathbb{D}(h_i)\big), \tag{6}$$

where $\mathcal{N}$ denotes the normalizing operator. The processor between the encoders and the decoder is the hierarchical multi-scale processor of HI-MGN, described next, with the latent modulation of Section 2.5 inserted before every block.

2.3. Hierarchical multi-scale processor

The processor arranges message passing over H coarse graphs in a V-cycle (Fig. 2a,b). It consists of $2H{+}1$ message passing stages, with $\{L_0, \ldots, L_{2H}\}$

denoting the number of blocks at each stage. The processor first performs L_0 message passes at the original level, after which the latent states are pooled to the coarser graph. At the next level, the latent vectors are refined for L_1 passes and pooled again. After the latent vectors are refined at the coarsest level, they are upsampled by the learned, geometry-aware interpolation network, merged with the corresponding skip states and refined once more on the way up. In this work, $H = 2$ with 4096 and 256 coarse nodes, and $\{L_s\} = \{4, 6, 8, 6, 4\}$, i.e., 28 blocks in total.

The coarse graphs are built by the FPS-Voronoi strategy of HI-MGN, which requires no externally supplied meshes (Algorithm 1). First, k seeds are chosen by greedy FPS on the reference coordinates, starting from a random node. A multi-source breadth-first search over the original mesh adjacency then assigns every node to its nearest seed in hops, giving graph-based Voronoi clusters. The coarse nodes are the seeds; their latent features are the mean of the latent states of their cluster, and their coordinates are inherited from the seeds. Two coarse nodes are connected when at least one fine-mesh edge crosses between their clusters, so the coarse connectivity follows the mesh topology and introduces no artificial connection between nearby but unconnected parts. For $H > 1$ the procedure is applied recursively. The coarse edge features are recomputed from the coarse node positions in the same eight-dimensional form, normalized with statistics of each level, and embedded by a separate edge encoder per level.

Upsampling lifts the coarse latents back to the finer level with a learned, geometry-aware GNN. A coarse-to-fine graph connects each fine node i to its own coarse seed and the neighbouring coarse nodes of that seed, forming a set of coarse source nodes $\mathcal{C}(i)$. Each source $c \in \mathcal{C}(i)$ sends a message built from the coarse latent h_c, the stored skip state s_i of the fine node, and the reference-configuration offset of i from the seed,

$$\mu_{ic} = \psi([h_c, s_i, p_i - p_c]), \tag{7}$$

and the fine latent state is reconstructed from the aggregated messages together with the skip state, then merged with the skip state by a linear map,

$$h_i^{\mathrm{up}} = \mathrm{MLP}\Big(\big[s_i, \textstyle\sum_{c\in\mathcal{C}(i)} \mu_{ic}\big]\Big), \qquad h_i = W_{\mathrm{skip}}\,[\,s_i,\, h_i^{\mathrm{up}}\,] + b_{\mathrm{skip}}. \tag{8}$$

The forward pass of the processor with the latent modulation is summarized in Algorithm 2. The modulation Mod and the part ζ_ℓ of the latent variable read at level ℓ are defined in Sections 2.4 and 2.5.

Algorithm 1 FPS–Voronoi coarsening strategy of HI-MGN [28].

Require: Fine mesh graph (V, E), node positions $\{p_i\}_{i \in V}$, number of coarse nodes k

FPS sampling

$s_1 = \text{random node}, \quad S = \{s_1\}$ ▷ S: set of FPS seeds

$d_i = \|p_i - p_{s_1}\|, \ \forall i \in V$

for $j = 2, \ldots, k$ **do**

  $s_j = \arg\max_{i \in V \setminus S} d_i$

  $S = S \cup \{s_j\}$

  $d_i = \min\big(d_i, \ \|p_i - p_{s_j}\|\big), \ \forall i \in V \setminus S$

end for

Voronoi partitioning

$C(i) = \arg\min_{a \in \{1,\ldots,k\}} \text{hop}(i, s_a), \ \forall i \in V$ ▷ C: Voronoi assignment map

▷ $\text{hop}(i, j)$: shortest path between nodes i and j in mesh edges

$\mathcal{C}_a = \{\, i \in V : C(i) = a \,\}, \ \forall a \in \{1, \ldots, k\}$

Coarse graph construction

$V_c = S$ ▷ V_c: Coarse nodes

$E_c = \{(C(i), C(j)) \mid (i, j) \in E, \ C(i) \neq C(j)\}$ ▷ E_c: Coarse edges

return $C, \ S \ (V_c, E_c)$

Because the first FPS seed is a random node, the hierarchy depends on a random seed. During training, two partitions with different seeds are precomputed for every sample, and one of them is drawn at every visit, so that the model does not fit one particular partition. Validation uses the first partition, and inference builds the hierarchy of every geometry with a fixed seed. The clusters are therefore random patches of similar size, not a decomposition into physical regions.

Algorithm 2 Hierarchical message-passing processor of HI-MGN-V with latent modulation.

Require: Fine latent features h; latent variable $\mathbf{z}$; graph edges $\{E^{(\ell)}\}_{\ell=0}^{H}$; coarsening maps $\{C^{(\ell)}, S^{(\ell)}\}_{\ell=0}^{H-1}$; message-passing depths $\{L_s\}_{s=0}^{2H}$

Downsampling

for $\ell = 0, \dots, H-1$ **do**
  for $j = 1, \dots, L_\ell$ **do**
    $h = \mathrm{Mod}_j\big(\mathrm{MP}_j,\ h,\ \zeta_\ell,\ E^{(\ell)}\big)$ ▷ Eq. (10)
  end for
  $h_{\mathrm{skip}}^{(\ell)} = h$
  $h = \mathrm{MeanPool}(h, C^{(\ell)})$
end for

Coarsest-level propagation

for $j = 1, \dots, L_H$ **do**
  $h = \mathrm{Mod}_j\big(\mathrm{MP}_j,\ h,\ \zeta_H,\ E^{(H)}\big)$
end for

Upsampling

for $\ell = H-1, \dots, 0$ **do**
  $h = \mathrm{Interpolate}\big(h,\ h_{\mathrm{skip}}^{(\ell)},\ C^{(\ell)},\ S^{(\ell)}\big)$ ▷ Eqs. (7)–(8)
  for $j = 1, \dots, L_{2H-\ell}$ **do**
    $h = \mathrm{Mod}_j\big(\mathrm{MP}_j,\ h,\ \zeta_\ell,\ E^{(\ell)}\big)$
  end for
end for

return h

2.4. Latent placement

The latent variable has up to two parts,

$$\mathbf{z}_g \in \mathbb{R}^{S\times D},\ S = H+1 = 3,\ D = 32; \qquad \mathbf{z}_\ell \in \mathbb{R}^{n_c\times C},\ n_c = 256,\ C = 4. \tag{9}$$

The global part holds one vector of length D per level, and the local part one vector of length C per coarsest cluster. Each fine node v has an ancestor $a_\ell(v)$ at level ℓ and a coarsest ancestor $a_H(v)$. The placement selects which parts exist and defines the part $\zeta_\ell(v)$ that node v reads at level ℓ (Table 3, Fig. 2d). Blocks at a level read that level's part, on the way down and on the way up. Where the latent variable acts states what the model assumes about the effect of ξ on the field.

- *Global.* The effect of ξ is summarized by a few variables shared by the whole structure, for example whether the shell buckles and how large the deformation is. The latent variable is one vector per level, read by every node.
- *Local.* The effect of ξ is regional: an imperfection at one place changes the response nearby. The latent variable is a field on the coarsest clusters, so a location can be written directly into it.
- *Hybrid.* Both: a shared variable and a regional field, sampled jointly.

In the shell problem ξ has only three random variables, but its effect is both global (buckled or not) and spatially structured (where the buckle lobes form and how far around the circumference they extend). A low-dimensional ξ does not imply that a global vector suffices, and a localized effect does not imply that a local field suffices [50, 62]. We therefore treat the placement as a modelling choice and study it as an ablation.

Placement	Parts	$\zeta_\ell(v)$ read at level ℓ	$\dim \zeta$	size of $\mathbf{z}$
global	$\mathbf{z}_g$	$\mathbf{z}_g[\min(\ell, S-1)]$	32	96
local	$\mathbf{z}_\ell$	$\mathbf{z}_\ell[a_H(v)]$	4	1024
hybrid	$\mathbf{z}_g, \mathbf{z}_\ell$	$[\,\mathbf{z}_\ell[a_H(v)] \mid \mathbf{z}_g[\min(\ell, S-1)]\,]$	36	1120

Table 3: Part of the latent variable read by node v at level ℓ. In the local and hybrid placements every node reads the regional vector of its coarsest cluster at *every* level, so the regional vector acts on the whole V-cycle instead of only at the coarsest level.

2.5. Latent modulation of the message passing blocks

Before each block B_j of Algorithm 2 (Fig. 2c), the latent variable scales and shifts the node features and sets how much of the block update is kept.

With h the node features entering the block and ζ the part of the latent variable read by the node,

$$(\gamma,\ \beta,\ g) = W_j\,\mathrm{SiLU}(\zeta) + b_j, \qquad \gamma, \beta, g \in \mathbb{R}^H, \tag{10}$$

$$\tilde{h} = h \odot (1 + \gamma) + \beta, \tag{11}$$

$$h' = h + g \odot \big(B_j(\tilde{h}) - \tilde{h}\big), \tag{12}$$

where $B_j(\tilde{h}) - \tilde{h}$ is the residual node update of Eqs. (4) and (5). The edge latent states are updated by the block as usual and are not modulated. W_j is initialized to zero and b_j to $(\gamma, \beta, g) = (0, 0, 1)$, so at initialization $h' = B_j(h)$: the network starts as the deterministic HI-MGN, and the latent variable has an effect only once training changes W_j. This feature-wise scale and shift is known as feature-wise linear modulation (FiLM) [71]. A similar form is used in diffusion transformers [72], there with a normalization step and a gate that starts at zero; here there is no normalization and the gate starts at one. The modulation costs $28 \times (\dim \zeta + 1) \times 3H$ parameters: 354,816 (global), 53,760 (local) and 397,824 (hybrid).

2.6. Posterior encoder

During training, the posterior encoder infers the latent variable from the FE result (Fig. 2e). The normalized field $\mathbf{y}$ and the node inputs are embedded by two MLPs and fused, and three message passing blocks on the fine mesh, without latent modulation, process the result (the encoder trunk). Two output branches read the trunk.

- *Global branch.* An attention-weighted average over the nodes of each geometry, followed by linear maps to the mean and log-variance $\mu_g, \log \sigma_g^2 \in \mathbb{R}^{S \times D}$.
- *Local branch.* Trunk features are mean-pooled into the coarsest clusters, two message passing blocks run on the coarsest graph, and linear maps give $\mu_\ell, \log \sigma_\ell^2 \in \mathbb{R}^{n_c \times C}$.

The global placement uses the first branch, the local placement the second and the hybrid both. The standard deviation is bounded below by 0.05. The latent variable is drawn as $\mathbf{z} = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, which lets the training gradient pass through the random draw [73].

2.7. Regularization of the latent variable

A regularization term keeps the inferred latent vectors close to a standard normal distribution $\mathcal{N}(0, I)$, so that they can later be generated without the FE result. The default is an MMD over the batch; a per-sample KL divergence is implemented as the alternative.

MMD over the batch. The MMD compares the set of inferred vectors $\{z_i\}_{i=1}^n$ in a batch with draws $\{e_j\}_{j=1}^n \sim \mathcal{N}(0, I)$ and is zero when the two sets have the same distribution. We use the estimate that includes the $i = i'$ terms,

$$\widehat{\mathrm{MMD}}^2 = \frac{1}{n^2}\sum_{i,i'} k(z_i, z_{i'}) + \frac{1}{n^2}\sum_{j,j'} k(e_j, e_{j'}) - \frac{2}{n^2}\sum_{i,j} k(z_i, e_j), \tag{13}$$

with a mixture of Gaussian kernels $k(a, b) = \frac{1}{5}\sum_m \exp\bigl(-\|a - b\|^2/(c_m h)\bigr)$, $c_m \in \{0.25, 0.5, 1, 2, 4\}$, and h the median squared cross distance, held without gradient [66, 64, 65]. For $\mathbf{z}_g$ the estimate is computed for each level vector over the batch and averaged over the levels. For $\mathbf{z}_\ell$ every coarsest cluster of every graph is one row, with at most 4096 rows subsampled. The rows of one geometry are correlated, so this term matches the distribution of the four values of a cluster, pooled over clusters; it says nothing about the correlation between clusters, which the prior of Section 2.8 has to reproduce. In the hybrid the two estimates are summed. MMD acts on the set of latent vectors and does not limit how much each vector says about its FE result, which suits a graph network that should use the latent variable.

KL per sample. The KL divergence of each sample's distribution $\mathcal{N}(\mu, \sigma^2)$ from $\mathcal{N}(0, I)$,

$$\mathcal{R}_{\mathrm{KL}} = \mathrm{mean}_{\mathrm{coords}}\Bigl[-\tfrac{1}{2}\bigl(1 + \log\sigma^2 - \mu^2 - \sigma^2\bigr)\Bigr], \tag{14}$$

is averaged over the components within each part and summed over the parts that exist.

2.8. Conditional flow-matching prior

At prediction time the latent variable is drawn from a distribution conditioned on the nominal input. It is learned by flow matching [74, 75]: a network v_θ learns a velocity field that moves Gaussian noise ϵ to the latent vectors seen in training along straight paths in a pseudo-time $t \in [0, 1]$. The

target is the standardized latent vector $\tilde{\mathbf{z}} = (\mathbf{z} - m)/s$ (Section 2.9). With one $t \sim U[0,1]$ per graph and $\sigma_{\min} = 10^{-4}$,

$$\mathbf{z}_t = \big(1 - (1-\sigma_{\min})t\big)\epsilon + t\,\tilde{\mathbf{z}}, \qquad \mathcal{L}_{\mathrm{FM}} = \mathbb{E}\,\big\|v_\theta(\mathbf{z}_t, t, c) - \big(\tilde{\mathbf{z}} - (1-\sigma_{\min})\epsilon\big)\big\|^2. \tag{15}$$

A sample is obtained by integrating $\dot{\mathbf{z}} = v_\theta$ from $t = 0$ to 1 with 30 steps of Heun's second-order method [76] and undoing the standardization. The pseudo-time enters as sines and cosines of 16 frequencies $2^k\pi$, $k = 0, \ldots, 15$ (32 values), denoted $\tau(t)$. The prior reads the nominal input through its own condition trunk: node and edge encoders and three message passing blocks of width 128 on the fine mesh, run once per geometry without $\mathbf{y}$.

Global placement. The condition trunk is reduced by an attention-weighted average to one vector $\kappa \in \mathbb{R}^{128}$ per geometry. The velocity of the 96 components of $\mathbf{z}_g$ is an MLP of $[\mathrm{vec}(\mathbf{z}_t) \mid \tau(t) \mid \kappa]$.

Local placement. The condition-trunk features are mean-pooled into the coarsest clusters, κ_v for $v \in V_H$, and the coarse edges are encoded. The node stream starts as $x_v = \mathrm{Lin}([\mathbf{z}_{\ell,t}(v) \mid \kappa_v])$ and passes through four message passing blocks on the coarsest graph, each modulated by the time embedding with the gated form of Eq. (10), initialized at the identity. A linear head, initialized at 0.01 of its default scale, gives $v_\ell(v) \in \mathbb{R}^C$.

Hybrid placement. The hybrid adds one vector ω per geometry that carries the global part through the same blocks:

$$\omega \leftarrow \mathrm{Lin}\big([\mathrm{vec}(\mathbf{z}_{g,t}) \mid \kappa]\big), \tag{16}$$

$$\text{before block } j: \quad \omega \leftarrow \omega + \mathrm{MLP}_j\big([\omega \mid \bar{x} \mid \tau(t)]\big), \qquad x_v \leftarrow x_v + \mathrm{Lin}_j(\omega), \tag{17}$$

$$v_g = \mathrm{Lin}([\omega \mid \bar{x}]), \tag{18}$$

where $\bar{x}$ is the mean of the node stream over the clusters. The maps Lin_j start at zero, so at initialization the regional part behaves as in the local placement. One pseudo-time per geometry drives both parts, and the loss is the sum of the regional and global flow-matching terms. The two parts are therefore generated together and exchange information at every velocity evaluation.

2.9. Loss and training

All input and output channels are standardized with statistics of the training split. With the predicted field $\hat{\mathbf{y}} = D(c, \mathbf{z})$ and $\mathbf{z}$ inferred from the FE result, the loss of the joint phase is

$$\mathcal{L} = \alpha \frac{1}{N} \sum_i \sum_q w_q \big(\hat{y}_{i,q} - y_{i,q}\big)^2 + \lambda \mathcal{R} + \beta \mathcal{L}_{\mathrm{PV}} + w \mathcal{L}_{\mathrm{FM}}, \tag{19}$$

with $\alpha = 1000$, $\beta = 10$ and $w = 1$, normalized feature weights w_q summing to one over the output channels q, and $(\mathcal{R}, \lambda) = (\widehat{\mathrm{MMD}}^2, 1)$ or $(\mathcal{R}_{\mathrm{KL}}, 10^{-3})$. $\mathcal{L}_{\mathrm{PV}}$ is the squared error of the per-graph peak-to-valley $\max_i \hat{y}_{i,z} - \min_i \hat{y}_{i,z}$ of the normalized axial displacement, an additional term that weights an extreme value of the field. The flow-matching term is trained on a separate draw from the encoder with μ and σ held fixed, so it does not change the encoder. The loss is a weighted sum chosen for the field, not the likelihood bound of a standard VAE.

Training has two phases (Algorithm 3). In the joint phase all networks train together. At epoch E_{freeze} the graph network and the encoder are frozen, and the standardization of the flow-matching target is computed once from the latent vectors that the frozen encoder infers over the training set,

$$m = \mathbb{E}[\mu], \qquad s = \sqrt{\mathrm{Var}(\mu) + \mathbb{E}[\sigma^2]}, \tag{20}$$

per component for $\mathbf{z}_g$ and per channel for $\mathbf{z}_\ell$. The prior then continues alone on $\mathcal{L}_{\mathrm{FM}}$ with a new optimizer and learning-rate schedule, in full precision and without a forward pass of the graph network; the standardization values are also copied into the exponential moving average (EMA) of its weights that is used for prediction. Fitting the prior to the latent vectors of a frozen encoder removes the moving target that a jointly trained prior has to follow, while the joint phase still gives the prior a good starting point. The model with the best validation score of the joint phase is kept, and the selection restarts for the prior-only phase. The validation score is the continuous ranked probability score (CRPS) of the normalized field, estimated per node and component with the fair ensemble estimator from 8 fields generated with the prior.

At prediction, for a new geometry, the hierarchy is built with the fixed seed, the condition trunk runs once, Gaussian noise is integrated through the velocity field with 30 Heun steps, the standardization is undone, and the graph network runs once per generated field. The integration acts on a 96-component vector (global) or on the 256 coarsest clusters (local and hybrid);

Algorithm 3 Training HI-MGN-V (any placement).

for epoch $= 1, \ldots, E_{\text{freeze}}$ **do** ▷ joint phase
  for batch $(c, \mathbf{y})$, partition drawn from the hierarchy variants **do**
    $(\mu, \sigma) \leftarrow q_\phi(c, \mathbf{y})$; $\mathbf{z} \leftarrow \mu + \sigma \odot \epsilon$
    $\hat{\mathbf{y}} \leftarrow D(c, \mathbf{z})$ ▷ blocks modulated by $\zeta_\ell(v)$, Eq. (10)
    $\mathcal{L} \leftarrow \alpha\,\mathrm{MSE} + \lambda\mathcal{R} + \beta\mathcal{L}_{\text{PV}} + w\,\mathcal{L}_{\text{FM}}(\mathrm{sg}[\mathbf{z}'])$ ▷ $\mathbf{z}'$: second encoder draw, held fixed
    update all parameters
  end for
end for
freeze D, q_ϕ; compute (m, s) over the training set; new optimizer
for epoch $= E_{\text{freeze}} + 1, \ldots, E$ **do** ▷ prior-only phase
  update p_θ on $\mathcal{L}_{\text{FM}}$ of standardized encoder draws
end for

the fine mesh is visited by the condition trunk once per geometry and by the graph network once per generated field. This is the main difference in cost from generators that iterate on a latent graph tied to the mesh, such as LDGN and SAR [41, 42].

2.10. Parameter count

Table 4 lists the parameter counts of the three placements. The deterministic V-cycle has 4,667,523 parameters; the graph networks of the three placements differ from it only by the modulation layers and, for the local and hybrid placements, by the coarse-level bookkeeping of the regional vector. Most of the difference between the placements comes from the encoder branches and the prior. The hybrid has 23% more parameters than the global placement, so a comparison of the default placements mixes placement with capacity.

Placement	total	graph network	(modulation)	encoder	prior
global	6,240,293	5,022,339	354,816	640,449	577,505
local	6,882,959	4,721,283	53,760	948,488	1,213,188
hybrid	7,652,657	5,065,347	397,824	973,385	1,613,925

Table 4: Parameter counts of HI-MGN-V with the 28-block V-cycle of Section 3.2 (MMD; the KL variants are identical). The deterministic HI-MGN with the same V-cycle has 4,667,523 parameters.

3. Numerical examples

In this section, we evaluate HI-MGN-V on a shell-buckling problem in which an unmeasured geometric imperfection decides whether a nominal shell buckles. HI-MGN-V is compared with the deterministic HI-MGN [28] and with our ports of LDGN and SAR [41, 42]. The deterministic HI-MGN uses the same V-cycle as HI-MGN-V and gives the mean-response reference. The hyperparameters of LDGN and SAR are set to the values of the original authors, and only those hyperparameters that the authors themselves vary between their examples are adapted. Within HI-MGN-V, the latent placement (global, local, hybrid) and the regularization (MMD, KL) are studied as ablations.

3.1. Axially compressed cylindrical shells with random imperfections

We consider thin cylindrical shells under axial compression. Figure 3 illustrates the configuration. The bottom rim is clamped in all six degrees of freedom, and the top rim is driven axially by an end shortening Δ at a fixed speed of $0.25\,\mathrm{mm/ms}$. The shell dimensions vary across samples, characterized by a radius $R \in [100, 200]\,\mathrm{mm}$, a thickness $t \in [1, 2]\,\mathrm{mm}$, a length $L \in [150, 300]\,\mathrm{mm}$ and an end shortening $\Delta \in [1, 3]\,\mathrm{mm}$, which are selected via Latin hypercube sampling with 150 geometries. The classical buckling load is $P_{cr} = 3.803\,Et^2$ and the classical shortening $d_{cr} = 0.60523\,tL/R$; the load ratio is $\lambda = \Delta/d_{cr}$. The shells are linear elastic steel without yield, discretized with structured quadrilateral shell elements, and solved with the explicit solver OpenRadioss. The dataset is summarized in Table 5.

Each geometry is solved 64 times, each time with a different Gaussian dimple in its initial shape (Fig. 4). The dimple has three random variables: its amplitude $a \sim \mathcal{N}(0, 0.4^2)\,\mathrm{mm}$, its angular position, uniform over the circumference, and its axial position, uniform at least 2σ away from both rims,

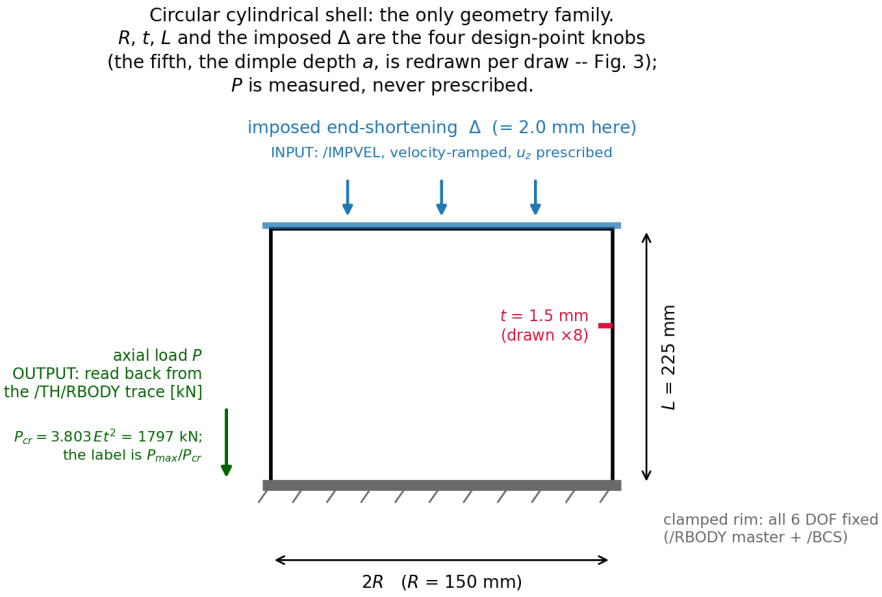


Figure 3: Boundary conditions, shown at the centre of the design box ($R = 150$, $t = 1.5$ (drawn $8\times$ thicker), $L = 225\,\mathrm{mm}$). The bottom rim is clamped in all six degrees of freedom. The top rim is driven axially by Δ at $0.25\,\mathrm{mm/ms}$. The reaction force is an output, not an input. $P_{cr} = 3.803\,Et^2$ is 1797 kN at the centre.

where the dimple width σ equals the classical axisymmetric half-wavelength l_c. The model sees only the nominal, imperfection-free cylinder, and the imperfection parameters are stored for diagnosis only. Depending on the dimple, the same nominal shell stays unbuckled or buckles.

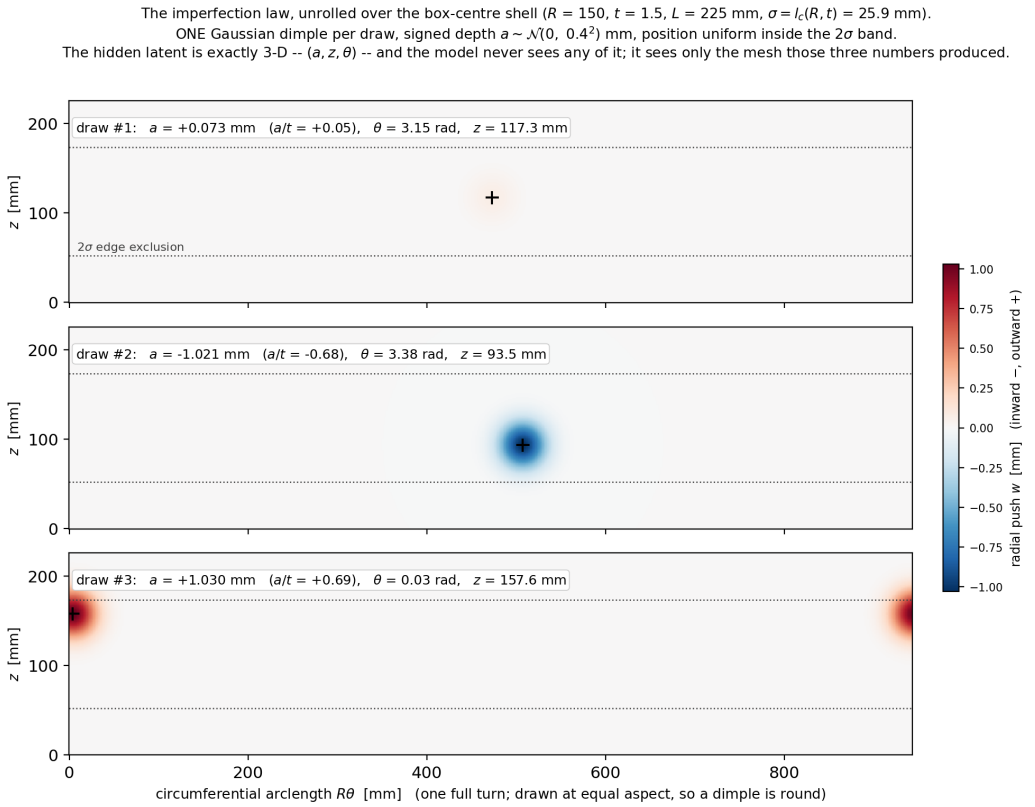


Figure 4: The unmeasured input ξ. Three samples on the unrolled centre shell ($a = +0.073, -1.021, +1.030$ mm). The dimple width is $\sigma = l_c$ (25.9 mm at the centre). Its centre is uniform inside the dashed band, 2σ away from both rims. The imperfection enters only the solver's initial geometry; the model input is the nominal cylinder.

Buckling indicator. A draw is counted as buckled when its peak displacement $s = \max_i \|\mathbf{u}_i\|$ exceeds 5.17 mm. The threshold lies between the largest unbuckled and the smallest buckled peak and agrees with the solver's buckling flag on 97.5% of the FE draws. Figure 6(b) shows that the 64 draws of one geometry near the critical load give a bimodal peak displacement, and Fig. 6(c) that the buckling probability goes from 0 to 1 over $\lambda \approx 0.6$–

Item	Value
Design variables	$R \sim U[100, 200]$, $t \sim U[1, 2]$, $L \sim U[150, 300]$, $\Delta \sim U[1, 3]$ (mm); 4-D Latin hypercube, 150 geometries
Imperfection ξ (not a model input)	64 samples per geometry; one Gaussian dimple, three random variables: amplitude $a \sim \mathcal{N}(0, 0.4^2)$ mm, uniform angle, axial centre uniform away from a 2σ band at each rim; width $\sigma = l_c = \pi\sqrt{Rt}/[12(1-\nu^2)]^{1/4}$
Material	linear elastic steel, no yield; $E = 210\,\text{GPa}$, $\nu = 0.3$, $\rho = 7.85 \times 10^{-6}\,\text{kg/mm}^3$
Elements, mesh	structured quadrilateral shells, 5 through-thickness integration points; $h \leq l_c/10$; 27,975–87,398 nodes in training (median 45,409)
Solver	OpenRadioss explicit; units kg–mm–ms; terminal kinetic-to-internal energy ratio median 0.034, maximum 0.176
Boundary conditions	bottom rim clamped (6 DOF); top rim axial velocity 0.25 mm/ms to Δ
Node features	coordinates; displacement u (output); thickness t and end shortening Δ (input-only conditions); node type
Split	by geometry, seed 42: 120 train / 15 validation / 15 test
Imperfection recipes	the 64 recipes (amplitude, angle, axial quantile) are shared by all 150 geometries; the out-of-range file reuses them for draws 0–63 and adds 192 fresh recipes (draws 64–255)

Table 5: Shell-buckling dataset. The imperfection parameters are stored for diagnosis only and are not model inputs.

1.05 across geometries. The scatter of the response is thus decided by the imperfection, and its mean is not a possible response.

Where the buckle forms. The dimple seeds the buckle but does not fix where it lies. In the buckled FE draws of the out-of-range geometries, the node of largest radial displacement lies within one dimple width σ of the dimple centre, in angle and in axial position, in only 5–27% of the draws of a geometry, against 1–5% for a position chosen at random. The radial lobes above 80% of the peak cover a median of 6–14 of 18 angular sectors, so a buckle usually spreads around much of the circumference. A buckled field therefore does not reveal its imperfection, and a generated field is compared with independent FE results of the same nominal design, not with a re-analysis of an imperfection read off the field.

3.2. Training and inference settings

For training, we consider a three-level hierarchy. Letting N be the number of nodes in the training mesh, HI-MGN-V and HI-MGN are configured with the hierarchy $[N, 4096, 256]$ and the block configuration $[4, 6, 8, 6, 4]$, i.e., 28 message passing blocks. Two hierarchy variants are precomputed per training sample. The latent variable uses $D = 32$ per level for $\mathbf{z}_g$ and $C = 4$ channels per coarsest cluster for $\mathbf{z}_\ell$. During training, random rotations about the cylinder axis and random reflections in x and y are applied to the training geometries, as the boundary conditions and the imperfection distribution are preserved under these transformations; no noise is added to the inputs. The remaining hyperparameters are kept identical across the placements: 360 epochs with the prior-only phase starting at epoch 240, a batch size of 16, a latent dimension of 128, the Adam optimizer with a learning rate of 10^{-4} and a weight decay of 10^{-4}, a linear warmup over 5 epochs followed by cosine annealing, gradient clipping at a norm of 3.0, an EMA of the weights with a factor of 0.99, and bfloat16 mixed-precision training. The prior-only phase restarts Adam with its own warmup and cosine schedule. Validation runs every 10 epochs on the 15 validation geometries, and the EMA weights are used for validation and inference. The full configuration is listed in Appendix Appendix A. All HI-MGN-V models are trained on a single NVIDIA B300 GPU.

For inference, we consider unseen geometries inside and outside the training range. For every FE draw of an evaluation geometry, the model generates 4 fields from the prior with the hierarchy built from a fixed seed. Since the

draws of a geometry share the nominal input, the generated fields of a geometry are pooled: 256 fields for a test geometry with 64 FE draws, and 1024 fields for an out-of-range geometry with 256 FE draws.

3.3. Compared models

HI-MGN. The deterministic HI-MGN uses the same inputs, hierarchy, V-cycle and training settings as HI-MGN-V, without the latent variable, and is trained by the MSE alone. It returns one field per geometry and serves as the mean-response reference.

LDGN and SAR. The two generative baselines are ports of the authors' released code [41, 42] into the same data pipeline. In both, a geometry is one system, and its FE draws take the place of the flow states of the original works. Both receive the same inputs and split as HI-MGN-V and generate the final displacement field; the imperfection parameters are never read. The models, their two training stages (a graph VAE, then a generative model in its latent space), their mesh coarsening and their samplers are unchanged. SAR uses the authors' wing configuration: a single-scale VAE of two layers and width 128 with a KL weight of 10^{-6}, and three scale-autoregressive iterations of 8 layers each with width 128, 64 slices and 8 heads, sampled with 20, 11 and 2 steps. LDGN uses a three-level VAE of depths 2, 2, 1 and width 126 with a one-channel latent field and a KL weight of 10^{-6}, and a denoiser of depths 1, 2, 2, 2 and width 128 with 1000 diffusion steps of a linear schedule, learned variances and a variational-bound weight of 10^{-3}, sampled with 50 steps. Smaller batches with gradient accumulation reproduce the authors' effective batch sizes, and both models train until the learning rate of the authors' plateau schedule falls below its stopping threshold; the epoch limits only cap the wall-clock time. The targets are scaled to $[-1, 1]$ by their training minimum and maximum, as in the original code; a variant with standardized targets is also run. Before the shell problem, both ports were trained on the authors' wing dataset and reproduce the published results: the median ratio of the field Wasserstein distance of SAR to that of the released model is 1.02, and LDGN reaches an R^2 of the ensemble mean of 0.9962 (released 0.9965) and a field Wasserstein distance of 2.11 (released 1.99).

HI-MGN-V ablations. The six HI-MGN-V variants are {global, local, hybrid} $\times$ {MMD, KL}. They share every other setting of Section 3.2; only the latent placement and the regularization (with its weight) differ. The hybrid placement with MMD is the reference configuration.

3.4. Evaluation sets and metrics

There are two evaluation sets. The test set has the 15 unseen geometries inside the design box with their 64 FE draws, which reuse the imperfection recipes of the training set. The out-of-range set has 8 geometries that each move one design variable of the box centre $(150, 1.5, 225, 2.0)$ outside the box (Fig. 5, Table 6), with 256 FE draws each; draws 0–63 reuse the training recipes and draws 64–255 use fresh ones. The validation geometries are used only for model selection.

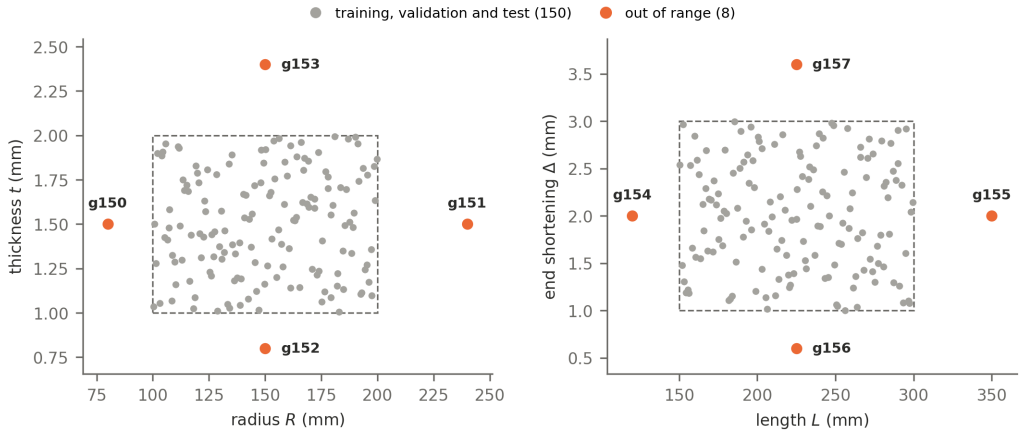


Figure 5: In-range (training, validation and test) and out-of-range geometries in the four design variables; the dashed rectangle is the design box. Each out-of-range geometry moves one variable from the box centre $(150, 1.5, 225, 2.0)$: R 80/240, t 0.8/2.4, L 120/350, Δ 0.6/3.6; each panel shows the four that move its variables.

shape	moved variable	λ	P_b	nodes
150	$R = 80$	0.783	0.41	46,224
151	$R = 240$	2.350	1.00	45,982
152	$t = 0.8$	2.754	1.00	85,968
153	$t = 2.4$	0.918	0.75	28,718
154	$L = 120$	2.754	1.00	24,852
155	$L = 350$	0.944	0.83	71,068
156	$\Delta = 0.6$	0.441	0.00	45,780
157	$\Delta = 3.6$	2.644	1.00	45,780

Table 6: Out-of-range geometries. λ: load ratio; P_b: FE buckling probability from draws 0–63.

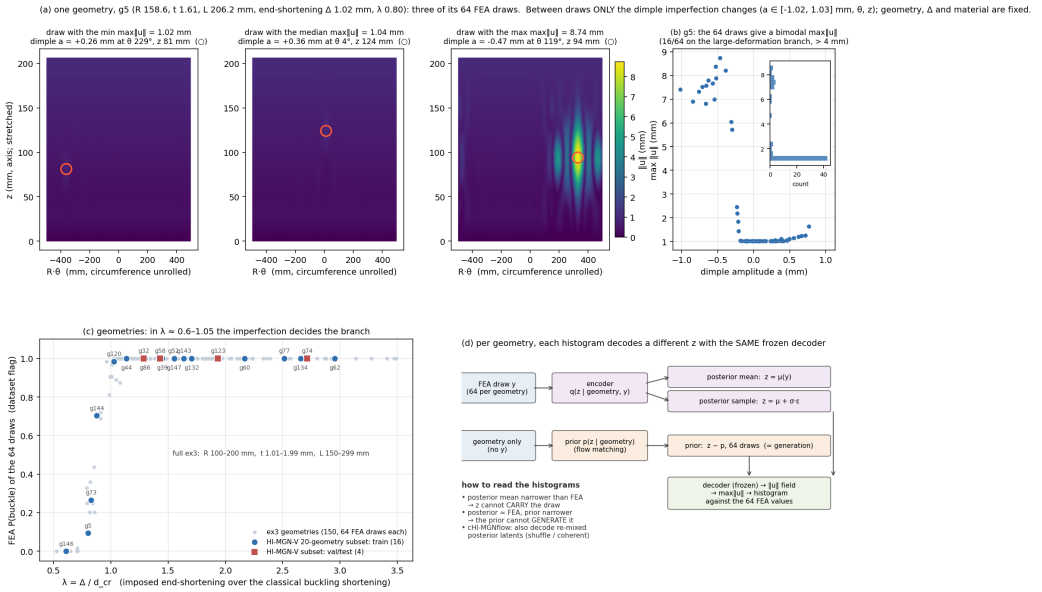


Figure 6: Evaluation setup. (a) Three of the 64 FE draws of geometry g5 ($\lambda = 0.80$); only the dimple changes between draws. (b) The 64 draws of g5 give a bimodal peak displacement. (c) FE buckling probability against λ for all 150 geometries. (d) Posterior-versus-prior check: the same trained graph network generates fields from latent vectors inferred from the FE results (posterior) and from latent vectors drawn from the prior, and each set is compared with the FE results.

Every metric is computed geometry by geometry, because the scatter within one geometry is bimodal and its shape changes with λ (Fig. 6). The primary quantity is the peak displacement $s = \max_i \|\mathbf{u}_i\|$ of each field, which contains both the axial shortening and the radial buckling displacement.

Wasserstein distance of the peak displacement. For one geometry, let F and $\hat{F}$ be the empirical distributions of s over the FE draws and over the generated fields, with means $m, \hat{m}$ and standard deviations $\sigma, \hat{\sigma}$. The one-dimensional 2-Wasserstein distance is computed exactly from the quantile functions and normalized by the FE standard deviation of that geometry. It decomposes exactly into a bias, a spread and a shape term,

$$\frac{W_2^2(\hat{F}, F)}{\sigma^2} = \underbrace{\left(\frac{\hat{m} - m}{\sigma}\right)^2}_{\text{bias}} + \underbrace{\left(\frac{\hat{\sigma}}{\sigma} - 1\right)^2}_{\text{spread}} + \underbrace{2\,\frac{\hat{\sigma}}{\sigma}\,(1 - \rho)}_{\text{shape}}, \tag{21}$$

where ρ is the correlation between the two quantile functions. The score of a set is the root mean square of W_2/σ over its geometries, reported $\times 100$. Even a model that samples from the FE distribution does not score zero with a finite number of draws. On the out-of-range set, the 256 FE draws of every geometry are therefore split at random into two halves of 128; the first half is scored against the second as if it were generated, which gives the value that FE results alone reach, and the model is scored with 128 of its generated fields against the same second half. Over 200 random splits, the FE-alone value has a median of 27 (5–95%: 20–34), and a model below this range cannot be distinguished from the FE scatter.

Further scalar scores. For s, we also report the CRPS of the generated ensemble against each FE draw, normalized by σ; the spread ratio $\hat{\sigma}/\sigma$ (1 is correct, below 1 is too little scatter); and the buckling-probability error $|P_{\text{model}} - P_{\text{FE}}|$ with the threshold of 5.17 mm. Results are shown per geometry as histograms of the generated peak displacements beside the FE histograms, not only as averaged scores.

Posterior versus prior. Before any comparison of models, the trained graph network generates two sets of fields for each geometry: one from latent vectors inferred from the FE results by the encoder (posterior), one from latent vectors drawn from the prior. If the posterior set matches the FE results and the prior set does not, the prior is at fault; if both fail, the graph network or

the encoder is. This check tells which part of the model a difference belongs to.

Field-level scores. For the out-of-range geometries, the generated fields are also compared with the FE fields as whole fields, on the same mesh. For a generated field $\hat{\mathbf{y}}$ and an FE result $\mathbf{y}$, $R^2(\hat{\mathbf{y}} \mid \mathbf{y}) = 1 - \sum_i \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|^2 / \sum_i \|\mathbf{y}_i - \bar{\mathbf{y}}\|^2$ and $d(\hat{\mathbf{y}}, \mathbf{y}) = (\frac{1}{N} \sum_i \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|^2)^{1/2}$. Since the cylinder, its boundary conditions and the imperfection distribution are axisymmetric, a turn of an FE result about the axis by a multiple of the angular mesh pitch is an FE result of the same design, and $R^2_{\circlearrowleft}$ takes the best of these turns. Sample realism (a) is measured in both directions,

$$A_{\text{real}} = \frac{1}{n} \sum_j \max_k R^2_{\circlearrowleft}(\hat{\mathbf{y}}_j \mid \mathbf{y}_k), \qquad A_{\text{cov}} = \frac{1}{K} \sum_k \max_j R^2_{\circlearrowleft}(\hat{\mathbf{y}}_j \mid \mathbf{y}_k), \tag{22}$$

which ask whether every generated field is close to some FE result and whether every FE result is close to some generated field, as precision and recall [77]; the first direction is the nearest-state R^2 of LDGN and SAR [41, 42]. Distribution match (b) is measured by the field Wasserstein distance

$$W_2^{\text{field}} = \Big(\min_{\pi} \frac{1}{n} \sum_j d(\hat{\mathbf{y}}_j, \mathbf{y}_{\pi(j)})^2 \Big)^{1/2}, \tag{23}$$

where π runs over the one-to-one assignments of n generated fields to n FE results and is found exactly by the Hungarian algorithm; no turn is taken out, since a model that puts every buckle at the same angle must fail (b). Both are reported beside the values that the FE results alone reach with the same number of samples, from the same split into halves of 128 [32, 78]. Out-of-range inference (c) is the repetition of (a) and (b) on the out-of-range set.

3.5. Results

To be completed.

4. Conclusions

To be completed.

Declaration of competing interest

To be completed.

Data availability

To be completed.

Appendix A. Hyperparameters

Group	Setting
Data	`input_var 3`, `output_var 3`, `cond_var 2`, `use_node_types`, `positional_features 4`, `edge_var 8`, `augment_geometry`, `std_noise 0`, `split_group_attr shape_idx`, `split_seed 42`
Training	single GPU; batch 16, `grad_accum_steps 1`; Adam, lr 10^{-4}, weight decay 10^{-4}; warm-up 5 epochs, then cosine annealing; gradient clip 3.0; 360 epochs; bfloat16 AMP; EMA 0.99; `training_seed 42`; `val_interval 10`; `best_by crps` with 8 prior draws
Graph network	`latent_dim 128`; `use_multiscale`; `coarsening_type voronoi_seedmean`; `multiscale_levels 2`; `voronoi_clusters 4096,256`; `mp_per_level 4,6,8,6,4`; `hierarchy_variants 2`; `hierarchy_seed 42` (inference)
Latent	`latent_type global|local|hybrid`; `vae_latent_dim 32` (3 levels); `latent_ch 4`; `vae_mp_layers 3`; `vae_local_mp_layers 2`; `vae_graph_aware`; `posterior_min_std 0.05`; gated FiLM in every block
Loss	`alpha_recon 1000` (MSE); `latent_reg mmd` ($\lambda = 1$) or `kl` (`kl_weight 1e-3`); `beta_aux 10` (peak-to-valley, axial channel); `prior_nll_weight 1`; `prior_grad_to_encoder 0`
Prior	conditional flow matching; condition trunk of 3 blocks, width 128; coarse prior of 4 blocks; 30 Heun steps; `prior_freeze_epoch 240`
Inference	4 generated fields per FE draw; spread statistic `mag_max` on channels 0–2

Table A.7: HI-MGN-V settings shared by the six variants (`configs/MeshGraphNets_Variational/hyperparameter_sweep/r8/config_train_hybrid_e360.txt` for the hybrid MMD reference). Only `latent_type` and `latent_reg` (with its weight) differ between the variants.

References

[1] D. Xiao, Error estimation of the parametric non-intrusive reduced order model using machine learning, Comput. Methods Appl. Mech. Eng. 355 (2019) 513–534. doi:10.1016/j.cma.2019.06.018.

[2] A. Moosavi, R. Ştefănescu, A. Sandu, Multivariate predictions of local reduced-order-model errors and dimensions, Int. J. Numer. Methods Eng. 113 (3) (2018) 512–533. doi:10.1002/nme.5624.

[3] S. Lee, H. Cho, H. Kim, S. Shin, Time-domain non-linear aeroelastic analysis via a projection-based reduced-order model, The Aeronautical Journal 124 (1281) (2020) 1798–1818.

[4] J. S. Hesthaven, S. Ubbiali, Non-intrusive reduced order modeling of nonlinear problems using neural networks, J. Comput. Phys. 363 (2018) 55–78. doi:10.1016/j.jcp.2018.02.037.

[5] S. Lee, K. Jang, H. Cho, H. Kim, S. Shin, Parametric non-intrusive model order reduction for flow-fields using unsupervised machine learning, Computer Methods in Applied Mechanics and Engineering 384 (2021) 113999. doi:10.1016/j.cma.2021.113999.

[6] A. T. Mohan, D. V. Gaitonde, A deep learning based approach to reduced order modeling for turbulent flow control using lstm neural networks, arXiv preprint arXiv:1804.09269 (2018).

[7] S. Lee, K. Jang, S. Lee, H. Cho, S. Shin, Parametric model order reduction by machine learning for fluid–structure interaction analysis, Engineering with Computers 40 (2024) 45–60. doi:10.1007/s00366-023-01782-2.

[8] S. Lee, S. Lee, K. Jang, H. Cho, S. Shin, Data-driven nonlinear parametric model order reduction framework using deep hierarchical variational autoencoder, Engineering with Computers 40 (2024) 2385–2400. doi:10.1007/s00366-023-01916-6.

[9] T. Kadeethum, F. Ballarin, D. O'Malley, Y. Choi, N. Bouklas, H. Yoon, Reduced order modeling with Barlow Twins self-supervised learning: Navigating the space between linear and nonlinear solution manifolds, arXiv preprint arXiv:2202.05460 (2022).

[10] K. Lee, K. T. Carlberg, Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders, J. Comput. Phys 404 (2020) 108973.

[11] A. Solera-Rico, C. S. Vila, M. Gómez, Y. Wang, A. Almashjary, S. Dawson, R. Vinuesa, β-variational autoencoders and transformers for reduced-order modelling of fluid flows, arXiv preprint arXiv:2304.03571 (2023).

[12] S. Lee, Development of a deep hierarchical variational autoencoder for efficient analysis including fluid–structure interaction, Ph.D. thesis, Seoul National University, Seoul (2024).

[13] S. Lee, S.-H. Kang, S. Lee, S. Shin, Physics-aware neural network-based parametric model-order reduction of the electromagnetic analysis for a coated component, Engineering with Computers 41 (2025) 785–799. doi:10.1007/s00366-024-02056-1.

[14] C. R. Qi, H. Su, K. Mo, L. J. Guibas, PointNet: Deep learning on point sets for 3D classification and segmentation, in: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 77–85. doi:10.1109/CVPR.2017.16.

[15] C. R. Qi, L. Yi, H. Su, L. Guibas, PointNet++: Deep hierarchical feature learning on point sets in a metric space, in: I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (Eds.), Advances in Neural Information Processing Systems, Vol. 30, Curran Associates, Inc., 2017.

[16] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, Nature Machine Intelligence 3 (2021) 218–229. doi:10.1038/s42256-021-00302-5.

[17] J. He, S. Koric, D. Abueidda, A. Najafi, I. Jasiuk, Geom-DeepONet: A point-cloud-based deep operator network for field predictions on 3D parameterized geometries, Comput. Methods Appl. Mech. Eng. 429 (2024) 117130.

[18] Z. Li, D. Z. Huang, B. Liu, A. Anandkumar, Fourier neural operator with learned deformations for PDEs on general geometries, Journal of

Machine Learning Research 24 (388) (2023) 1–26.
URL https://jmlr.org/papers/v24/23-0064.html

[19] Z. Li, N. Kovachki, C. Choy, B. Li, J. Kossaifi, S. Otta, M. A. Nabian, M. Stadler, C. Hundt, K. Azizzadenesheli, et al., Geometry-informed neural operator for large-scale 3D PDEs, in: Advances in Neural Information Processing Systems, Vol. 36, Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023, pp. 35836–35854.

[20] H. Wu, H. Luo, H. Wang, J. Wang, M. Long, Transolver: A fast transformer solver for PDEs on general geometries, arXiv preprint arXiv:2402.02366 (2024).

[21] B. Alkin, M. Bleeker, R. Kurle, T. Kronlachner, R. Sonnleitner, M. Dorfer, J. Brandstetter, AB-UPT: Scaling neural CFD surrogates for high-fidelity automotive aerodynamics simulations via anchored-branched universal physics transformers, arXiv preprint arXiv:2502.09692 (2025).

[22] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, P. W. Battaglia, Learning mesh-based simulation with graph networks, in: International Conference on Learning Representations, 2021.
URL https://arxiv.org/abs/2010.03409

[23] M. Fortunato, T. Pfaff, P. Wirnsberger, A. Pritzel, P. Battaglia, MultiScale MeshGraphNets, iCML 2022 AI for Science Workshop; arXiv:2210.00612 (2022).
URL https://arxiv.org/abs/2210.00612

[24] Y. Cao, M. Chai, M. Li, C. Jiang, Efficient learning of mesh-based physical simulation with bi-stride multi-scale graph neural network, in: Proceedings of the 40th International Conference on Machine Learning, Vol. 202 of Proceedings of Machine Learning Research, 2023, pp. 3541–3558.
URL https://proceedings.mlr.press/v202/cao23a.html

[25] M. A. Nabian, C. Liu, R. Ranade, S. Choudhry, X-meshgraphnet: Scalable multi-scale graph neural networks for physics simulation, arXiv preprint arXiv:2411.17164 (2024).

[26] M. M. Iparraguirre, I. Alfaro, D. Gonzalez, E. Cueto, Meshgraphnet-transformer: Scalable mesh-based learned simulation for solid mechanics, arXiv preprint arXiv:2601.23177 (2026).

[27] Y.-Y. Yu, J. Choi, J. Park, K. Lee, N. Park, PIORF: Physics-informed Ollivier-Ricci flow for long-range interactions in mesh graph neural networks, arXiv preprint arXiv:2504.04052 (2025).

[28] S. Lee, D.-H. Park, T. Bang, S.-H. Kang, HI-MeshGraphNets: Efficient and accurate mesh-based physics learning with hierarchical multi-scale graph neural networks, manuscript (2026).

[29] J. W. Hutchinson, W. T. Koiter, Postbuckling theory, Applied Mechanics Reviews 23 (1970) 1353–1366.

[30] X. Zhu, B. Sudret, Emulation of stochastic simulators using generalized lambda models, SIAM/ASA Journal on Uncertainty Quantification 9 (4) (2021) 1345–1380. doi:10.1137/20M1337302.
URL https://arxiv.org/abs/2007.00996

[31] X. Zhu, B. Sudret, Stochastic polynomial chaos expansions to emulate stochastic simulators, International Journal for Uncertainty Quantification 13 (2) (2023) 31–52. doi:10.1615/Int.J.UncertaintyQuantification.2022042912.
URL https://arxiv.org/abs/2202.03344

[32] N. Lüthen, S. Marelli, B. Sudret, A spectral surrogate model for stochastic simulators computed from trajectory samples, Computer Methods in Applied Mechanics and Engineering 406 (2023) 115875. doi:10.1016/j.cma.2022.115875.

[33] X. Shen, N. Meinshausen, Engression: extrapolation through the lens of distributional regression, Journal of the Royal Statistical Society Series B: Statistical Methodology (2025).
URL https://arxiv.org/abs/2307.00835

[34] Y. Yang, P. Perdikaris, Conditional deep surrogate models for stochastic, high-dimensional, and multi-fidelity systems, Computational Mechanics 64 (2019) 417–434. doi:10.1007/s00466-019-01718-y.
URL https://arxiv.org/abs/1901.04878

[35] N. E. Silionis, T. Liangou, K. N. Anyfantis, Deep learning-based surrogate models for spatial field solution reconstruction and uncertainty quantification in structural health monitoring applications, Computers

& Structures 301 (2024) 107462. doi:10.1016/j.compstruc.2024.107462.
URL https://www.sciencedirect.com/science/article/abs/pii/S0045794924001913

[36] J. Han, B. Bahmani, A multimodal conditional mixture model with distribution-level physics priors, International Journal for Numerical Methods in Engineering (2026) e70346doi:10.1002/nme.70346.
URL https://arxiv.org/abs/2602.10451

[37] R. Azimi, Y. Grinberg, D.-X. Xu, O. Liboiron-Ladouceur, Gen-Fab: A variation-aware generative model for predicting fabrication variations in nanophotonic devices, Structural and Multidisciplinary Optimization-ArXiv:2603.11505 (2026).
URL https://arxiv.org/abs/2603.11505

[38] J. Zheng, C. Jahnke, W. Chen, GUST: Quantifying free-form geometric uncertainty of metamaterials using small data, Journal of Mechanical DesignArXiv:2506.12051 (2026).
URL https://arxiv.org/abs/2506.12051

[39] F. Hendriks, O. Rokoš, M. Doškář, M. G. D. Geers, V. Menkovski, Equivariant flow matching for symmetry-breaking bifurcation problems, Machine Learning and the Physical Sciences Workshop, NeurIPS 2025; arXiv:2509.03340 (2025).
URL https://arxiv.org/abs/2509.03340

[40] A. Zimmel, F. Hendriks, M. Holzleitner, F. Sestak, M. Weichselbaumer, V. Menkovski, J. Brandstetter, Bi-FORK: Generative modeling of high-dimensional bifurcating systems, arXiv:2610.12449v1 (2026).
URL https://arxiv.org/abs/2610.12449v1

[41] M. Lino, T. Pfaff, N. Thuerey, Learning distributions of complex fluid simulations with diffusion graph networks, in: International Conference on Learning Representations, 2025.
URL https://arxiv.org/abs/2504.02843

[42] M. Lino, N. Thuerey, One scale at a time: Scale-autoregressive modeling for fluid flow distributions, arXiv:2604.11403v1 (2026).
URL https://arxiv.org/abs/2604.11403v1

[43] X. Song, T. Yu, Physically-informed flow matching with graph neural networks for complex fluid dynamics, Proceedings of the AAAI Conference on Artificial Intelligence 40 (2) (2026) 1024–1032. doi:10.1609/aaai.v40i2.37072.
URL https://ojs.aaai.org/index.php/AAAI/article/view/37072

[44] R. Molinaro, S. Lanthaler, B. Raonić, et al., Generative AI for fast and accurate statistical computation of fluids, arXiv:2409.18359 (2024).
URL https://arxiv.org/abs/2409.18359

[45] M. Lienen, D. Lüdke, J. Hansen-Palmus, S. Günnemann, From zero to turbulence: Generative modeling for 3D flow simulation, in: International Conference on Learning Representations, 2024.
URL https://arxiv.org/abs/2306.01776

[46] Q. Liu, N. Thuerey, Uncertainty-aware surrogate models for airfoil flow simulations with denoising diffusion probabilistic models, AIAA Journal 62 (8) (2024).
URL https://arxiv.org/abs/2312.05320

[47] J. Oskarsson, T. Landelius, M. P. Deisenroth, F. Lindsten, Probabilistic weather forecasting with hierarchical graph neural networks, in: Advances in Neural Information Processing Systems, Vol. 37, 2024.
URL https://arxiv.org/abs/2406.04759

[48] I. Price, A. Sanchez-Gonzalez, F. Alet, et al., Probabilistic weather forecasting with machine learning, Nature 637 (2025) 84–90. doi:10.1038/s41586-024-08252-9.

[49] F. Alet, I. Price, A. El-Kadi, D. Masters, S. Markou, T. R. Andersson, J. Stott, R. Lam, M. Willson, A. Sanchez-Gonzalez, P. Battaglia, Skillful joint probabilistic weather forecasting from marginals, arXiv:2506.10772 (2025).
URL https://arxiv.org/abs/2506.10772

[50] S. Park, A. Zhou, H. Kim, A. B. Farimani, Generative neural operators through diffusion last layer, arXiv:2602.04139v2; ICML 2026 (2026).
URL https://arxiv.org/abs/2602.04139v2

[51] K. Preechakul, N. Chatthee, S. Wizadwongsa, S. Suwajanakorn, Diffusion autoencoders: Toward a meaningful and decodable representation,

in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022.
URL https://arxiv.org/abs/2111.15640

[52] P. Dahlinger, N. Freymuth, M. Volpp, T. Hoang, G. Neumann, Latent task-specific graph network simulators, arXiv:2311.05256 (2023).
URL https://arxiv.org/abs/2311.05256

[53] B. Nath, M. Lakhadive, A. Sharma, B. Bhowmik, A physics-aware variational graph autoencoder for joint modal identification with uncertainty quantification, arXiv:2604.11237 (2026).
URL https://arxiv.org/abs/2604.11237

[54] T. Park, M.-Y. Liu, T.-C. Wang, J.-Y. Zhu, Semantic image synthesis with spatially-adaptive normalization, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019.
URL https://arxiv.org/abs/1903.07291

[55] P. Zhu, R. Abdal, Y. Qin, P. Wonka, SEAN: Image synthesis with semantic region-adaptive normalization, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 5104–5113.
URL https://arxiv.org/abs/1911.12861

[56] Z. Tan, M. Chai, D. Chen, J. Liao, Q. Chu, B. Liu, G. Hua, N. Yu, Diverse semantic image synthesis via probability distribution modeling, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 7962–7971.
URL https://arxiv.org/abs/2103.06878

[57] S. A. A. Kohl, B. Romera-Paredes, K. H. Maier-Hein, D. J. Rezende, S. M. A. Eslami, P. Kohli, A. Zisserman, O. Ronneberger, A hierarchical probabilistic U-Net for modeling multi-scale ambiguities, arXiv:1905.13077 (2019).
URL https://arxiv.org/abs/1905.13077

[58] C. F. Baumgartner, K. C. Tezcan, K. Chaitanya, A. M. Hötker, U. J. Muehlematter, K. Schawkat, A. S. Becker, O. Donati, E. Konukoglu, PHiSeg: Capturing uncertainty in medical image segmentation, in: Medical Image Computing and Computer Assisted Intervention (MICCAI), 2019.
URL https://arxiv.org/abs/1906.04045

[59] G. Yang, X. Huang, Z. Hao, M.-Y. Liu, S. Belongie, B. Hariharan, PointFlow: 3d point cloud generation with continuous normalizing flows, in: IEEE/CVF International Conference on Computer Vision, 2019.
URL https://arxiv.org/abs/1906.12320

[60] X. Zeng, A. Vahdat, F. Williams, Z. Gojcic, O. Litany, S. Fidler, K. Kreis, LION: Latent point diffusion models for 3d shape generation, in: Advances in Neural Information Processing Systems, Vol. 35, 2022.
URL https://arxiv.org/abs/2210.06978

[61] Y. Xie, S. J. Pan, Global and local topology-aware graph generation via dual conditioning diffusion, in: International Conference on Learning Representations, 2026.
URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/af2bb2b2280d36f8842e440b4e275152-Abstract-Conference.html

[62] K. Masuki, Y. Ashida, Renormalization group flow matching for scalable local generative modeling, arXiv:2608.23696 (2026).
URL https://arxiv.org/abs/2608.23696

[63] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, A. Lerchner, β-VAE: Learning basic visual concepts with a constrained variational framework, in: International Conference on Learning Representations, 2017.

[64] S. Zhao, J. Song, S. Ermon, InfoVAE: Balancing learning and inference in variational autoencoders, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 33, 2019, pp. 5885–5892.
URL https://arxiv.org/abs/1706.02262

[65] I. Tolstikhin, O. Bousquet, S. Gelly, B. Schölkopf, Wasserstein auto-encoders, in: International Conference on Learning Representations, 2018.
URL https://arxiv.org/abs/1711.01558

[66] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, A. Smola, A kernel two-sample test, Journal of Machine Learning Research 13 (2012) 723–773.

[67] P. Ghosh, M. S. M. Sajjadi, A. Vergari, M. Black, B. Schölkopf, From variational to deterministic autoencoders, in: International Conference

on Learning Representations, 2020.
URL https://arxiv.org/abs/1903.12436

[68] B. Dai, D. Wipf, Diagnosing and enhancing VAE models, in: International Conference on Learning Representations, 2019.
URL https://arxiv.org/abs/1903.05789

[69] P. W. Battaglia, et al., Relational inductive biases, deep learning, and graph networks, arXiv:1806.01261 (2018).
URL https://arxiv.org/abs/1806.01261

[70] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, P. Battaglia, Learning to simulate complex physics with graph networks, in: Proceedings of the 37th International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 8459–8468.

[71] E. Perez, F. Strub, H. de Vries, V. Dumoulin, A. Courville, FiLM: Visual reasoning with a general conditioning layer, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 32, 2018.
URL https://arxiv.org/abs/1709.07871

[72] W. Peebles, S. Xie, Scalable diffusion models with transformers, in: IEEE/CVF International Conference on Computer Vision, 2023.
URL https://arxiv.org/abs/2212.09748

[73] D. P. Kingma, M. Welling, Auto-encoding variational bayes, in: International Conference on Learning Representations, 2014.
URL https://arxiv.org/abs/1312.6114

[74] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le, Flow matching for generative modeling, in: International Conference on Learning Representations, 2023.
URL https://arxiv.org/abs/2210.02747

[75] X. Liu, C. Gong, Q. Liu, Flow straight and fast: Learning to generate and transfer data with rectified flow, in: International Conference on Learning Representations, 2023.
URL https://arxiv.org/abs/2209.03003

[76] T. Karras, M. Aittala, T. Aila, S. Laine, Elucidating the design space of diffusion-based generative models, in: Advances in Neural Information Processing Systems, Vol. 35, 2022.
URL https://arxiv.org/abs/2206.00364

[77] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, T. Aila, Improved precision and recall metric for assessing generative models, in: Advances in Neural Information Processing Systems, Vol. 32, 2019.

[78] R. Kansal, A. Li, J. Duarte, N. Chernyavskaya, M. Pierini, B. Orzari, T. Tomei, Evaluating generative models in high energy physics, Physical Review D 107 (2023) 076017.
URL https://arxiv.org/abs/2211.10295